

Los 5 Apóstoles

Arquitectura actual del sistema · repo: chinchudo · PC MIDI Center

Dashboard interno de inteligencia comercial, social listening y respuestas asistidas por IA. El sistema combina un dashboard web en la nube con agentes que corren en una PC local y controlan navegadores reales. Todo se sincroniza a través de una única base de datos en Supabase.

Las 4 capas del sistema

1 · Vercel — dashboard web (Next.js 14)

Producción · login por contraseña (AUTH_SECRET) · 14 páginas server-rendered · server actions

Operación social / · /opportunities · /personas · /brands · /products · /knowledge · /prompts · /monitoring	Crecimiento SEO /landings · /leads · /distribution · /geo · /analytics · /informe · /logins	API routes /api/leads · /events · /click · /nurture · /unsubscribe · /stats · /analytics
---	---	--

▼ Prisma ORM

2 · Supabase Postgres — fuente única de verdad

Pooled (app + agentes) · Direct (migraciones) · 20+ tablas

Social Opportunity · Response · PublishingLog · MonitoredSource · Channel	Conocimiento Brand · Product · Persona · KnowledgeBase · Objection · PromptVersion	SEO / Leads Landing · LeadMagnet · Lead · NurtureStep · DistributionPiece · GeoAudit · TrackingEvent
Sistema AppSetting (incluye AGENT_RELAY_URL) · SystemLog		

▼ el relay escribe la URL del túnel en AppSetting

3 · Tu PC (Windows) — agentes locales

Semi-automático · cloudflared expone el relay → Vercel le manda pedidos de publicación (fire-and-forget, 202)

agent-relay.mjs (3099) + cloudflared /publish · /login-status · /accounts · /debug · /health — auth Bearer token	orchestrator.py listen · monitor · daily · draft · export · healthcheck → corre social-listen.py por cada fuente activa	swarm.py research · generate · build-landings · nurture · distribution · geo-audit · conversion
NSTBrowser / Dolphin (CDP) 5 perfiles = 5 personas · auto-asignación de cuenta por canal · máx 3 browsers · extractors: youtube/reddit/facebo ok/instagram/x/tiktok/linkedin		

▼ servicios externos

4 · Servicios externos

OpenRouter

DeepSeek / Gemini — 3
variantes de borrador (SHORT ·
TECHNICAL ·
CONVERSATIONAL) y
resúmenes

SMTP nurturing

secuencia de emails a leads +
tracking de apertura / click

Serper / DuckDuckGo

investigación de keywords y
competidores para landings +
GEO

Pipeline A — Social listening + respuestas asistidas

El corazón del sistema. Dos pipelines independientes comparten la misma base de datos.

- 1 **Detección.** En /monitoring se cargan fuentes (MonitoredSource: canal + query + cuenta + límite). El orchestrator.py monitor las lee, auto-asigna qué perfil/persona usa cada canal (criterio: que la cuenta tenga el canal permitido y sea la menos usada recientemente) y limita a 3 navegadores simultáneos por vuelta; el resto queda diferido.
- 2 **Extracción.** social-listen.py se conecta vía CDP (WebSocket crudo, sin Selenium) a un navegador real de NSTBrowser/Dolphin y corre el extractor del canal. Sobre cada ítem aplica filtros: on-topic (keywords de dominio MIDI/audio + exclusiones tipo 'midi skirt'), idioma (español vs inglés), antigüedad (descarta comentarios viejos) y 'accionable' (descarta emojis/tags/elogios cortos sin pregunta). Lo que pasa va a data/social-listen-intake.jsonl.
- 3 **Importación.** import-opportunities.mjs lee el JSONL, re-clasifica intención/prioridad, detecta marca, deduplica y crea Opportunity con estado NEW.
- 4 **Generación IA.** 'Generar respuestas' arma un prompt con persona + marca + producto del catálogo + KnowledgeBase + Objection + prompt activo, llama a OpenRouter y guarda 3 variantes. Hay rate-limit, reintentos con backoff y fallback a generador local si la IA falla. Reglas duras: nunca nombrar la tienda, hablar como usuario real, siempre mencionar un producto, nunca cerrar con pregunta.
- 5 **Aprobación.** Fede edita la mejor variante y la aprueba (estado APPROVED).
- 6 **Publicación.** Vercel no puede abrir navegadores, así que publishViaAgent hace POST /publish al relay en la PC (su URL pública vive en AppSetting.AGENT_RELAY_URL, actualizada automáticamente por el túnel cloudflared). El relay responde 202 y procesa en background: publish-response.mjs chequea anti-spam (cap diario por cuenta + separación mínima entre comentarios), publisher.py publica con el navegador real, marca la oportunidad como PUBLISHED y auto-descarta las oportunidades hermanas del mismo post para no comentar dos veces en el mismo lugar.
- 7 **Seguimiento.** PublishingLog registra URL, cuenta y resultado; las que necesitan respuesta quedan en FOLLOW_UP.

Pipeline B — Crecimiento SEO / GEO (swarm.py)

research (keywords) → generate (landings HTML con IA) → build-landings (deploy estático a blog.pcmidicenter.com) → captura de Lead vía API → nurture (secuencia de emails SMTP con tracking) → distribution (piezas para redes) → geo-audit (mide si las IAs recomiendan a PC MIDI) → conversion (analiza qué landings convierten). Todo persiste en Supabase y se visualiza en /landings, /leads, /geo y /analytics.

Lo que ata todo

- **Supabase es la única fuente de verdad** — tanto Vercel (Prisma/TS) como los agentes (Python y scripts Node) leen y escriben las mismas tablas.

- **El relay + cloudflared es el puente** que permite que un dashboard en la nube dispare acciones de navegador en tu máquina.
- **NSTBrowser da las 5 identidades** (una persona por perfil): Profe, Productor, Baterista, Kressmer y Cazador. Eso hace que las respuestas parezcan de usuarios reales distintos.
- Cada corrida deja un **reporte JSON** en reports/ y los errores de IA / rate-limit van a la tabla SystemLog.

Notas de revisión

- En el README el proyecto figura como 'Los 5 Apóstoles' / pcmidi-suite, pero se está subiendo al repo 'chinchudo'. Confirmar que es a propósito.
- agents/accounts.json y .env contienen credenciales reales y NO deberían terminar en GitHub. Conviene verificar que .gitignore los excluya antes de cualquier push.